Höhere technische Bundeslehranstalt
und Bundesfachschule
im Hermann Fuchs Bundesschulzentrum

PowerShell

Grundlagen und Aufgaben

Erstellt von:

Michel Rögl-Brunner, BEd

13. April 2026

Inhaltsverzeichnis

1 Grundlagen **2**
1.1 Was ist PowerShell? 2
1.2 Cmdlets und Namensschema 2
1.3 Hilfe und Dokumentation 2
1.4 Pipeline und Objekte 3

2 Kernbefehle in der Praxis **4**
2.1 Dateien und Verzeichnisse 4
2.2 Prozesse und Services 4
2.3 Ausgabe, Variablen und Export 4

3 Aufgabenserie 1: Prozesse und Services **6**
3.1 Hinweise zu typischen Lösungswegen 6

4 Aufgabenserie 2: Dateien, Skripte und Daten **7**
4.1 Empfohlene Cmdlets für Serie 2 7

5 Aufgabenserie 3: Fortgeschrittene Automatisierung **9**
5.1 Zusatzanforderungen für Serie 3 10
5.2 Empfohlene Cmdlets für Serie 3 10

6 Beispieldateien und Download **11**

7 Abgabe **12**

1 Grundlagen

1.1 Was ist PowerShell?

PowerShell ist eine Kommandozeile und Skriptsprache von Microsoft zur Verwaltung von Windows-Systemen. Im Unterschied zu klassischen Shells arbeitet PowerShell objektbasiert: Befehle geben nicht nur Text aus, sondern .NET-Objekte mit Eigenschaften und Methoden.

Dadurch lassen sich Ergebnisse gezielt filtern, sortieren und weiterverarbeiten. Viele Verwaltungsaufgaben wie Prozess-, Service- oder Dateiverwaltung sind damit strukturiert und gut automatisierbar.

1.2 Cmdlets und Namensschema

Die meisten PowerShell-Befehle heissen **Cmdlets** und folgen dem Schema `Verb-Noun`, zum Beispiel:

- `Get-Process`
- `Get-Service`
- `Sort-Object`
- `Export-Csv`

Dieses einheitliche Schema erleichtert das Lernen, weil ähnliche Aufgaben immer nach dem gleichen Muster gelöst werden.

1.3 Hilfe und Dokumentation

Für neue Befehle ist die integrierte Hilfe besonders wichtig:

- `Get-Help <Befehl>`: zeigt die Hilfe zum Befehl
- `Get-Help <Befehl> -Examples`: zeigt nur Beispiele
- `Get-Command`: listet verfügbare Befehle auf

Mit diesen Befehlen können unbekannte Cmdlets schnell verstanden und korrekt verwendet werden.

1.4 Pipeline und Objekte

Mit der Pipeline `|` wird die Ausgabe eines Befehls an den nächsten weitergegeben. In PowerShell werden dabei Objekte statt reiner Textzeilen übergeben.

Beispiel:

- `Get-Process | Sort-Object CPU -Descending | Select-Object -First 10`

Hier werden Prozesse geladen, nach CPU-Zeit sortiert und auf die ersten `10` Einträge begrenzt.

2 Kernbefehle in der Praxis

2.1 Dateien und Verzeichnisse

Wichtige Befehle für den Dateisystemzugriff sind:

- `Get-ChildItem`: listet Dateien und Ordner auf
- `New-Item`: erstellt Dateien oder Verzeichnisse
- `Copy-Item`: kopiert Dateien oder Ordner
- `Get-Content`: liest Dateiinhalt
- `Set-Content`: schreibt Inhalt in Dateien

2.2 Prozesse und Services

Für laufende Prozesse und Windows-Dienste sind folgende Cmdlets zentral:

- `Get-Process`: Prozesse anzeigen
- `Stop-Process`: Prozess beenden
- `Get-Service`: Services anzeigen
- `Start-Service` und `Stop-Service`: Service starten oder stoppen

2.3 Ausgabe, Variablen und Export

PowerShell unterstützt verschiedene Ausgabeformen:

- Variablen: `$P = ...`
- Textausgabe in Datei: `Out-File`
- CSV-Export: `Export-Csv`
- XML-Export: `Export-Clixml`

- HTML-Ausgabe: `ConvertTo-Html`

Diese Techniken sind wichtig, um Ergebnisse zu dokumentieren und in anderen Programmen weiterzuverwenden.

3 Aufgabenserie 1: Prozesse und Services

1. Erstelle eine Liste aller Prozesse, absteigend nach CPU-Nutzung sortiert.
2. Erzeuge eine Liste der Top-`10`-Prozesse basierend auf der CPU-Zeit.
3. Weise der Variable `$P` die Ausgabe aus Aufgabe 2 zu.
4. Speichere den Inhalt von `$P` in der Datei `A4.txt`. Exportiere den Inhalt von `$P` zusätzlich in eine CSV-Datei und in eine XML-Datei.
5. Erzeuge eine Liste aller Services und sortiere diese nach der Eigenschaft `Status`.
6. Erzeuge eine Liste aller Services und gib nur die Eigenschaften `Name` und `Status` in roter Schrift aus.
7. Erzeuge eine Liste aller Services und färbe die Ausgabe per `if`-Abfrage:
   - Rot bei `Stopped`/gestoppt
   - Grün bei `Running`/gestartet
8. Konvertiere die Ausgabe von `Get-Service` in HTML.

3.1 Hinweise zu typischen Lösungswegen

Folgende Befehle sind für diese Aufgaben besonders nützlich:

- `Get-Process | Sort-Object CPU -Descending`
- `Select-Object -First 10`
- `Out-File A4.txt`
- `Export-Csv .\A4.csv -NoTypeInformation`
- `Export-Clixml .\A4.xml`
- `Get-Service | Sort-Object Status`
- `Write-Host ... -ForegroundColor Red/Green`
- `Get-Service | ConvertTo-Html | Out-File services.html`

4 Aufgabenserie 2: Dateien, Skripte und Daten

1. **Dateien auflisten**
   Schreibe ein Skript, das alle Dateien im Verzeichnis `C:\Windows\System32` auflistet und Dateigrösse sowie Erstellungsdatum anzeigt.

2. **Verzeichnis erstellen**
   Entwickle ein Skript, das unter `C:\Temp\` ein neues Verzeichnis erstellt, falls es noch nicht existiert. Der Verzeichnisname soll auf aktuellem Datum und Uhrzeit basieren.

3. **Textdatei erstellen und schreiben**
   Erstelle eine Textdatei und speichere darin eine benutzerdefinierte Nachricht.

4. **Inhalt einer Datei lesen**
   Lies den Inhalt einer bestehenden Textdatei und gib ihn in der Konsole aus.

5. **Dateien kopieren**
   Kopiere alle `.txt`-Dateien von einem Quellverzeichnis in ein Zielverzeichnis.

6. **Systeminformationen abrufen**
   Zeige grundlegende Systeminformationen an, z. B. Computername, Betriebssystem und installierte RAM-Grösse.

7. **Benutzerdefinierte Funktion erstellen**
   Schreibe eine Funktion, die zwei Zahlen als Eingabe erhält und deren Summe zurückgibt. Rufe diese Funktion in einem Skript auf.

8. **CSV-Datei verarbeiten**
   Erstelle eine CSV-Datei mit Dummy-Daten im Format `Vorname;Nachname;Geburtsdatum` (mindestens 5 Einträge). Lies die CSV ein und gib die Benutzer nach Alter sortiert aus.

4.1 Empfohlene Cmdlets für Serie 2

- `Get-ChildItem`
- `New-Item -ItemType Directory`
- `Test-Path`

- `Set-Content` und `Get-Content`
- `Copy-Item`
- `Get-ComputerInfo`
- `Import-Csv`
- `Sort-Object`

5 Aufgabenserie 3: Fortgeschrittene Automatisierung

1. **Log-Analyse mit Fehlerbericht**
   Erstelle ein Skript, das eine Logdatei einliest und die Vorkommen von `ERROR`, `WARNING` und `INFO` zählt. Speichere das Ergebnis zusätzlich als CSV-Bericht mit Zeitstempel.
   Input-Datei: `BEISPIELDATEIN/POWERSHELL/Aufgabe-Serie-3-Log-Analyse/app.log`
   Direktlink: `https://raw.githubusercontent.com/michelroegl-brunner/HTL-Braunau/ma`

2. **Service-Monitor mit Protokollierung**
   Überwache eine definierte Liste kritischer Dienste (z. B. `Spooler`, `wuauserv`, `BITS`). Prüfe den Status und schreibe für jeden Dienst eine Zeile in eine Logdatei. Wenn ein Dienst gestoppt ist, soll dies deutlich markiert werden.

3. **Geplantes Backup mit Rotation**
   Sichere ein Quellverzeichnis in ein Zielverzeichnis mit Zeitstempel im Ordnernamen. Behalte nur die letzten 5 Backups und lösche ältere Sicherungen automatisch.

4. **Datei-Integrity-Check**
   Erstelle Prüfsummen (`SHA256`) für alle Dateien in einem Verzeichnis und speichere diese in einer Referenzdatei. Schreibe ein zweites Skript, das bei einem späteren Lauf Änderungen erkennt (neu, gelöscht, verändert) und übersichtlich ausgibt.

5. **Parameterisiertes Admin-Skript**
   Entwickle ein Skript mit Parametern `-Mode`, `-Path` und `-WhatIf`. Je nach Modus soll das Skript Dateien auflisten, archivieren oder löschen. Ungültige Parameter müssen mit einer verständlichen Fehlermeldung abgefangen werden.

6. **WMI/CIM-Inventarisierung mehrerer Rechner**
   Lies Rechnernamen aus einer Textdatei ein und sammle pro Rechner Systemdaten (OS, RAM, CPU, letzter Boot-Zeitpunkt) mit CIM. Erstelle daraus einen Gesamtbericht als CSV und als HTML-Tabelle.
   Input-Datei: `BEISPIELDATEIN/POWERSHELL/Aufgabe-Serie-3-CIM/rechnerliste.txt`
   Direktlink: `https://raw.githubusercontent.com/michelroegl-brunner/HTL-Braunau/ma`

5.1 Zusatzanforderungen für Serie 3

- Verwende in jedem Skript `try/catch` für Fehlerbehandlung.
- Nutze `param(...)` und sinnvolle Standardwerte.
- Gib am Ende einen klaren Erfolgs- oder Fehlerstatus aus (`exit 0`/`exit 1`).
- Trenne Logik in wiederverwendbare Funktionen.

5.2 Empfohlene Cmdlets für Serie 3

- `Get-Content`, `Select-String`, `Group-Object`
- `Get-Service`, `Start-Service`, `Out-File`
- `Copy-Item`, `Remove-Item`, `Get-ChildItem`
- `Get-FileHash`, `Compare-Object`
- `Get-CimInstance`, `Invoke-Command`
- `Export-Csv`, `ConvertTo-Html`

6 Beispieldateien und Download

Wenn fuer eine Aufgabe Beispieldateien benoetigt werden, liegen diese im Repository unter:

- `BEISPIELDATEIN/POWERSHELL/<AUFGABE>/...`
- `BEISPIELDATEIN/LINUX/<AUFGABE>/...`

Beispiele fuer diese Ausarbeitung:

- `BEISPIELDATEIN/POWERSHELL/Aufgabe-1/services.txt`
- `BEISPIELDATEIN/POWERSHELL/Aufgabe-2/A4.txt`
- `BEISPIELDATEIN/POWERSHELL/Aufgabe-2/A4.csv`
- `BEISPIELDATEIN/POWERSHELL/Aufgabe-3/benutzer.csv`
- `BEISPIELDATEIN/POWERSHELL/Aufgabe-Serie-3-Log-Analyse/app.log`
- `BEISPIELDATEIN/POWERSHELL/Aufgabe-Serie-3-CIM/rechnerliste.txt`

Download direkt ueber GitHub:

- Einzeldatei: Datei oeffnen, `Raw` klicken, danach speichern.
- Ganze Sammlung: `https://github.com/michelroegl-brunner/HTL-Braunau/archive/ref`
- Raw-Schema: `https://raw.githubusercontent.com/michelroegl-brunner/HTL-Braunau`

7 Abgabe

Die Ausarbeitung ist als **PDF-Datei** abzugeben.

Zu jeder Aufgabe ist ein Screenshot der Ausführung (Befehl und Ausgabe) zu dokumentieren.

Neu verwendete Cmdlets sollen in kurzen Stichworten erklärt werden.

Bitte lade deine fertige PDF in die entsprechende **Teams-Aufgabe** hoch.